

L2-2.4. Splunk: Data Manipulation

Task 1 - Introduction

Data processing, parsing, and manipulation in Splunk are crucial for extracting meaningful insights and enabling effective analysis of machine-generated data. Correctly parsed data allows Splunk to extract fields and values, making them searchable, structured, and ready for analysis. From a security perspective, these capabilities are particularly valuable in identifying and responding to security threats, investigating incidents, and monitoring system health. This is because well-structured data enables analysts to correlate events across multiple sources, detect anomalies more accurately, and create actionable detections or alerts.

Task 2 - Scenario Briefing

In this scenario, you will play the role of a SOC analyst at MSSP Cybertees Ltd. One of your clients requires ingesting event data into Splunk from a custom source. Splunk must be properly configured to parse and transform the logs appropriately. In this scenario, you will address the following issues.

- **Event Breaking:** Configure Splunk to correctly identify and separate individual events during ingestion
- **Multi-Line Event Parsing:** Configure Splunk to properly handle logs that span multiple lines
- **Sensitive Data Masking:** Mask sensitive information in log data to comply with PCI DSS (Payment Card Industry Data Security Standard)
- **Field Extraction:** Extract relevant fields from your client's event data

Splunk is installed in the default `/opt` directory, and you will be working with scripts located in `/home/ubuntu/Downloads/scripts` directory. Switch over to the root user with `sudo su` once you have access.

Answer the questions below

| What is the full file path to the Splunk directory?

/opt/splunk

How many executables are available in the /scripts directory?

3

Task 3 - How Splunk Processes Data

Splunk identifies timestamps, breaks events apart, assigns source types, and extracts key fields that analysts rely on. This is controlled through Splunk's configuration system and files. Let's examine how data is parsed below.

Splunk supports a wide range of common formats, including CSV, JSON, XML, and raw text. Knowing the structure of your data helps you choose, or create, the right source type and ensures Splunk applies the correct parsing rules.

The `sourcetype` in Splunk is one of the most important settings during data ingestion, as it represents the type of data and how to manage timestamps, line breaks, and field extractions. Splunk comes with many built-in source types for popular log sources, as seen below, but you can also define custom source types.

Name	Actions	Category	App
__singleline	Edit Clone		system
_json JavaScript Object Notation format. For more information, visit http://json.org/	Edit Clone	Structured	system
access_combined National Center for Supercomputing Applications (NCSA) combined format HTTP web server logs (can be generated by apache or other web servers)	Edit Clone	Web	system
access_combined_wcookie	Edit Clone		system
access_common	Edit Clone		system
ActiveDirectory	Edit Clone		system
anaconda	Edit Clone		system
anaconda_syslog	Edit Clone		system
apache_error Error log format produced by the Apache web server (typically error_log on 'nix systems)	Edit Clone	Web	system

The `inputs.conf` file instructs Splunk on what data to ingest, where it originates, how to collect it, and what basic metadata to assign, such as `sourcetype`, `host`, or `source`. Check out the following file locations and the example provided below. It is worth noting that apps created in Splunk will also have their own configuration files.

- `/opt/splunk/etc/system/default` This is Splunk's default configuration file location; however, these files are not intended for editing and will be overwritten when updates are made.
- `/opt/splunk/etc/system/local` This location contains custom changes and takes priority over the default file. Create or edit files here.

In this example, we instruct Splunk to monitor a specific log file, specifying both the index and source type.

```
[monitor:///path/to/your/data]
disabled = false
index = your_index
sourcetype = your_sourcetype
```

The `props.conf` file defines data parsing settings for specific source types or data sources within Splunk. In the following example, we reference the source type specified in `inputs.conf`, tell Splunk to treat every line as a new event, and define the time format.

```
[your_sourcetype]
SHOULD_LINEMERGE = false
TIME_FORMAT = %Y-%m-%d %H:%M:%S
```

Defining Field Extractions: You can define custom field extractions in `props.conf` by specifying the source type, field extraction name, and a custom regular expression, for example. In the next task, we will cover another Splunk configuration file that provides a more efficient solution for field extractions.

```
[your_sourcetype]
EXTRACT-fieldname1 = regular_expression1
EXTRACT-fieldname2 = regular_expression2
```

Using the Splunk UI: Although this room focuses on creating and editing Splunk configuration files from the command line, Splunk also provides a user-friendly web interface for managing many of the same settings. From the UI,

you can define source types, create field extractions, manage lookups, and configure additional parsing options without manually editing configuration files.

Field aliases Edit or add one or more aliases to field names	+ Add new
Calculated fields Edit or add one or more calculated fields	+ Add new
Field extractions View and edit all field extractions. Add new field extractions and update permissions.	+ Add new
Field transformations Edit or add transformations for field extractions that use a transform.	+ Add new
Sourcetype renaming Rename a source type. Multiple source types can share the same name.	+ Add new
Workflow actions Edit or add workflow actions	+ Add new

Answer the questions below

Which configuration file would you edit to instruct Splunk on what data to ingest?

inputs.conf

What is the full path to Splunk's default configuration files?

/opt/splunk/etc/system/default

Task 4 - Exploring Configuration Files

In this room, we will be making changes in the `/local` directory.

Configuration Files `transforms.conf`

- **Purpose:** Allows you to define data transformations and enrichments on indexed events
- **Example:** Perhaps you want to create a `username` field from the following log line. You could specify the custom field extraction, regular expression, and new field name in the configuration file `User john.doe logged in from 10.0.0.5`

```
[extract_username]
REGEX = User\s+(\S+)\s+logged
FORMAT = username:.$1
```

`fields.conf`

- **Purpose:** Defines how custom or extracted fields behave in Splunk
- **Example:** Below, `fields.conf` references the `transforms.conf` entry by specifying how to handle our new `username` field

```
[username]
INDEXED = true
```

`indexes.conf`

- **Purpose:** Manages the configuration of indexes in Splunk, including storage, retention policies, and access control
- **Example:** Suppose you want to create a new index named `my_index` with specific settings

```
[my_index]
homePath = $SPLUNK_DB/my_index/db
coldPath = $SPLUNK_DB/my_index/colddb
thawedPath = $SPLUNK_DB/my_index/thaweddb
maxTotalDataSizeMB = 100000
```

`outputs.conf`

- **Purpose:** Specifies the destination and settings for sending indexed data to various outputs, such as remote Splunk instances or third-party systems
- **Example:** You may choose to forward your indexed data to a remote Splunk indexer

```
[tcpout:group1]
server = 10.10.10.100:9997
```

Stanzas In Splunk: Splunk configuration files are organized into sections called **stanzas**, which define how Splunk should handle a specific component of its configuration. Every stanza begins with a text label in square brackets `[]`, as shown in the previous examples. This label identifies what the settings entered apply to. Depending on the configuration file, a stanza might represent:

- An index `indexes.conf`
- A sourcetype `props.conf`
- A field transformation `transforms.conf`
- An input source `inputs.conf`
- An output group `outputs.conf`

Answer the questions below

Which Splunk configuration file is used to define output destinations for indexed data?

`outputs.conf`

Take a look at the example `transforms.conf` entry below. What line would you add to `fields.conf` to begin defining the custom field?

```
[extract_sourceip]
REGEX = User\s+(\S+)\s+logged
FORMAT = SourceIP::$1
[SourceIP]
```

Task 5 - Creating a Splunk App

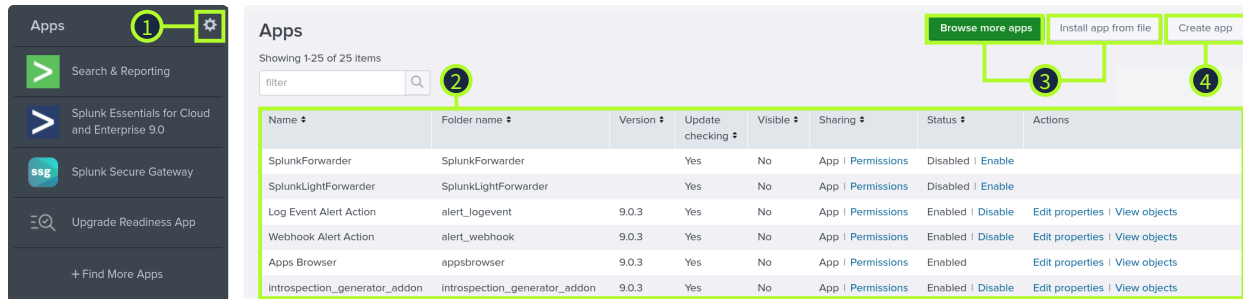
Firing Up Splunk: Before we can begin building our application, we must get Splunk running.

- Head to `/opt/splunk/bin` directory and run `./splunk start`
- Access the Splunk Dashboard at `MACHINE_IP:8000`

Creating Your Application: Once you have accessed the Splunk Dashboard

1. Click the `Manage Apps` cogwheel

2. This page shows the default installed Splunk apps
3. From here, you can [Browse more apps](#) or [Install app from file](#)
4. We want to click [Create app](#) to get started building our application



Next, we need to fill in the details about the app we want to create. Feel free to follow the naming conventions used below and name your app [DataApp](#). After filling out the information, click [Save](#). Your new app will be placed in the [/opt/splunk/etc/apps](#) directory of our virtual machine.

Name

Give your app a friendly name for display in Splunk Web.

Folder name *

This name maps to the app's directory in \$SPLUNK_HOME/etc/apps/.

Version

App version.

Visible ☐ No ☒ Yes

Only apps with views should be made visible.

Author

Name of the app's owner.

Description

Enter a description for your app.

Template

These templates contain example views and searches.

Upload asset No file selected.

Can be any html, js, or other file to add to your app.

Upon saving, you will now see your new application listed on the **Apps** page.

1. Your new application and relevant information
2. App permissions and status
3. App actions Click **Launch App**

Showing 1-25 of 26 items

filter

25 per page

< Prev 1 2 Next >

Name	Folder name	Version	Update checking	Visible	Sharing	Status	Actions
DataApp	DataApp	0.0.1	Yes	Yes	App Permissions	Enabled Disable	Launch app Edit properties View objects
SplunkForwarder	SplunkForwarder		Yes	No	App Permissions	Disabled Enable	
SplunkLightForwarder	SplunkLightForwarder		Yes	No	App Permissions	Disabled Enable	
Log Event Alert Action	alert_logevent	9.0.3	Yes	No	App Permissions	Enabled Disable	Edit properties View objects
Webhook Alert Action	alert_webhook	9.0.3	Yes	No	App Permissions	Enabled Disable	Edit properties View objects
Apps Browser	appsbrowser	9.0.3	Yes	No	App Permissions	Enabled	Edit properties View objects
introspection_generator_addon	introspection_generator_addon	9.0.3	Yes	No	App Permissions	Enabled Disable	Edit properties View objects
journald_input	journald_input		Yes	No	App Permissions	Enabled Disable	Edit properties View objects
Home	launcher		Yes	Yes	App Permissions	Enabled	Launch app Edit properties View objects
learned	learned		Yes	No	App Permissions	Enabled Disable	Edit properties View objects

Getting Familiar With the App Directory: Back on the command line, go ahead and run `ls /opt/splunk/etc/apps` to list the directory contents. You should see your newly created `DataApp` listed first.

List Splunk Apps

```
root@tryhackme:/opt/splunk/etc/apps# ls /opt/splunk/etc/apps
DataApp                                splunk-dashboard-studio
SplunkForwarder                       splunk_archiver
SplunkLightForwarder                 splunk_assist
alert_logevent                       splunk_essentials_9_0
alert_webhook                        splunk_gdi
...
```

Let's now take a closer look at our newly created `DataApp` directory.

Data App Folders

```
root@tryhackme:/opt/splunk/etc/apps# cd /opt/splunk/etc/apps/
DataApp
root@tryhackme:/opt/splunk/etc/apps/DataApp# ls
bin  default  local  metadata
```

- `bin` Contains custom scripts or executables used by the app
- `default` Stores the app's default configuration files
- `local` Stores modified configuration files
- `metadata` Contains permission metadata for the app

Generating Logs With Python: We now know that the `bin` directory contains any scripts that the app will use. Let's create a Python script that generates sample logs.

- Head to the `/bin` directory
- Create a new script with `echo 'print("This is a sample log...")' > samplelogs.py`
- Test your script by running `python3 samplelogs.py`

Log Generator Script

```
root@tryhackme:/opt/splunk/etc/apps/DataApp# cd bin
root@tryhackme:/opt/splunk/etc/apps/DataApp/bin# echo 'print
("This is a sample log...")' > samplelogs.py
root@tryhackme:/opt/splunk/etc/apps/DataApp/bin# python3 samp
lelogs.py
This is a sample log...
```

Creating Inputs: For Splunk to ingest the logs from our script, it needs to know the location of the script. To do this, we must create an `inputs.conf` file for the newly created app. Head to `/local` in your app directory, use your favorite text editor to create `inputs.conf`, and paste the following lines.

```
[script:///opt/splunk/etc/apps/DataApp/bin/samplelogs.py]
INDEX = main
SOURCETYPE = testing
HOST = test
INTERVAL = 5
```

The entry above takes the output from `samplelogs.py` and sends it to the `main` Splunk index every five seconds using our specified `sourcetype` and `host`. Go ahead and restart Splunk now, using `/opt/splunk/bin/splunk restart` for the changes to take effect.

Verifying Log Ingestion: Now that we have our script set up and `inputs.conf` configured, you can head back to your Splunk instance to verify log ingestion.

1. Verify you are using your newly created `DataApp`
2. Search `index = main`
3. Set the time range to `All time (real time)`
4. Check the result of your query

New Search Save As Create Table View Close

1 index = main 2 All time (real-time) 3

27 of 27 events matched No Event Sampling Job Smart Mode

Events (27) Patterns Statistics Visualization

Format Timeline Zoom Out Zoom to Selection Deselect 1 second per column

List Format 20 Per Page 4 Prev 1 2 Next

	Time	Event
>	11/26/25 8:05:46.000 AM	This is a sample log... host = tryhackme source = test_log sourcetype = testing
>	11/26/25 8:05:41.000 AM	This is a sample log... host = tryhackme source = test_log sourcetype = testing

SELECTED FIELDS
a host 1
a source 1
a sourcetype 1

Answer the questions below

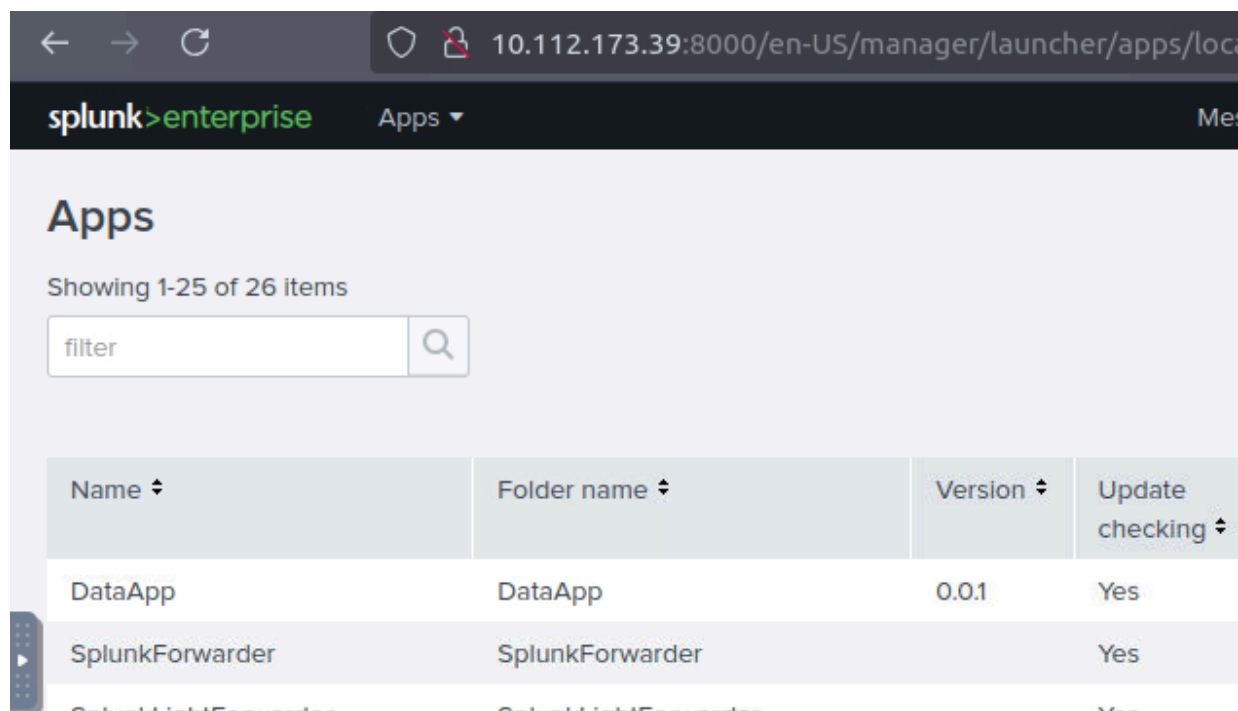
When Splunk is first installed, how many default applications are available?

25

What is the full path to your newly created application directory?

/opt/splunk/etc/apps/DataApp

```
root@tryhackme:/opt/splunk/etc/apps# ls
DataApp                               search
SplunkForwarder                      splunk-dashboard-studio
SplunkLightForwarder                 splunk_archiver
alert_logevent                       splunk_assist
alert_webhook                        splunk_essentials_9_0
appsbrowser                          splunk_gdi
inputs.conf                          splunk_httpinput
introspection_generator_addon         splunk_instrumentation
journald_input                       splunk_internal_metrics
launcher                             splunk_metrics_workspac
learned                              splunk_monitoring_conso
legacy                               splunk_rapid_diag
python_upgrade_readiness_app          splunk_secure_gateway
sample_app                           user-prefs
root@tryhackme:/opt/splunk/etc/apps# cd DataApp
root@tryhackme:/opt/splunk/etc/apps/DataApp#
```



Task 6 - Configuring Event Boundaries

Understanding the Events: In this task, we will be utilizing the `vpnlogs` executable. Let's copy over the executable to our app's `/bin` directory with the command below.

- `cp /home/ubuntu/Downloads/scripts/vpnlogs /opt/splunk/etc/apps/DataApp/bin/`

Ensure you're in the `/bin` directory and run the script with `./vpnlogs`. Based on the output, we can see three critical pieces of information that we will want to keep track of.

- User**: The user who connected
- Server**: The server the user connected to
- Action**: Connect to or Disconnect from the server

```
User: John Doe, Server: Server B, Action: DISCONNECT
User: Bob Johnson, Server: Server A, Action: CONNECT
User: John Doe, Server: Server D, Action: CONNECT
User: Emily Davis, Server: Server C, Action: CONNECT
User: Bob Johnson, Server: Server B, Action: CONNECT
```

```
User: Alice Smith, Server: Server C, Action: DISCONNECT
User: John Doe, Server: Server D, Action: CONNECT
User: Alice Smith, Server: Server C, Action: CONNECT
User: Michael Brown, Server: Server B, Action: DISCONNECT
```

Ingesting VPN Logs: Now that we have an understanding of the log type we want to ingest, let's get them into Splunk. We will follow a similar process to the one used in the previous task. We want to add an entry to `inputs.conf` in `/local` to point to the `vpnlogs` executable. Go ahead and add this entry now. Similar to the previous task, we instruct Splunk to run the script every five seconds and ingest the logs into the `main` index, specifying the `sourcetype` and `host`. After adding the entry, restart Splunk with `/opt/splunk/bin/splunk restart`.

```
[script:///opt/splunk/etc/apps/DataApp/bin/vpnlogs]
INDEX = main
SOURCETYPE = vpn_logs
HOST = vpn_server
INTERVAL = 5
```

Checking Out the Logs: Head back to your app in the Splunk instance and enter the search `index = main sourcetype = vpn_logs` and set your time range to `All time (real time)`. We can now see that the vpn logs are being ingested into Splunk, but there is an issue. Splunk is unable to determine the event boundaries, and each event includes ten lines from the vpn logs.

1 index = main sourcetype = vpn_logs

All time (real-time)

13 of 13 events matched No Event Sampling

Events (13) Patterns Statistics Visualization

Format Timeline Zoom Out Zoom to Selection Deselect

1 second per column

List Format 20 Per Page

< Hide Fields All Fields

SELECTED FIELDS

- host 1
- source 1
- sourcetype 1

INTERESTING FIELDS

- index 1
- punct 1
- splunk_server 1
- timestamp 1

+ Extract New Fields

Time Event

12/1/25 3:43:21.000 AM User: Emily Davis, Server: Server D, Action: DISCONNECT
User: John Doe, Server: Server B, Action: CONNECT
User: Alice Smith, Server: Server A, Action: CONNECT
User: John Doe, Server: Server B, Action: CONNECT
User: Michael Brown, Server: Server C, Action: DISCONNECT
Show all 10 lines
host = vpn_server source = /opt/splunk/etc/apps/DataApp/bin/vpnlogs sourcetype = vpn_logs

12/1/25 3:43:16.000 AM User: Emily Davis, Server: Server A, Action: DISCONNECT
User: John Doe, Server: Server D, Action: CONNECT
User: Emily Davis, Server: Server C, Action: CONNECT
User: John Doe, Server: Server C, Action: CONNECT
User: Alice Smith, Server: Server C, Action: DISCONNECT
Show all 10 lines
host = vpn_server source = /opt/splunk/etc/apps/DataApp/bin/vpnlogs sourcetype = vpn_logs

Ten Lines Per Event

Fixing Event Boundaries: As discussed in the previous tasks, we can specify the processing properties of our indexed events using the `props.conf` file. We will create a regular expression to instruct Splunk on how to identify the end of each event. If you review the sample events generated when we ran the script, you can see that all events end with either `CONNECT` or `DISCONNECT`.

REGULAR EXPRESSION

10 matches (168 steps, 605µs)

/(CONNECT|DISCONNECT)/gm

TEST STRING

User: Emily Davis, Server: Server C, Action: DISCONNECT

User: John Doe, Server: Server B, Action: DISCONNECT

User: Bob Johnson, Server: Server B, Action: DISCONNECT

User: Emily Davis, Server: Server D, Action: CONNECT

User: Alice Smith, Server: Server D, Action: CONNECT

User: Alice Smith, Server: Server A, Action: DISCONNECT

User: Bob Johnson, Server: Server C, Action: DISCONNECT

User: John Doe, Server: Server D, Action: DISCONNECT

User: John Doe, Server: Server B, Action: DISCONNECT

User: Michael Brown, Server: Server E, Action: CONNECT

Creating Your Configuration File: Next, we will need to create the `props.conf` configuration file.

- Navigate to `/opt/splunk/etc/apps/DataApp/local`
- Use your favorite text editor to create `props.conf`
- Paste in the following entry and restart Splunk with `/opt/splunk/bin/splunk restart`

```
[vpn_logs]
SHOULD_LINEMERGE = false
MUST_BREAK_AFTER = (CONNECT|DISCONNECT)
```

With the above entry, we are telling Splunk to use the `vpn_logs` source type, not to merge log lines since each line represents a singular event, and to recognize the beginning of a new event after `CONNECT` or `DISCONNECT`.

Verifying the Changes: Now that the configurations have been made, head back to your DataApp and run the query `index = main sourcetype = vpn_logs`. Be sure to set the time to `All time (real time)`. As shown in the screenshot below and in your Splunk instance, the event boundaries are now clearly defined, and Splunk recognizes each log line as a unique event. Nice job!

The screenshot shows the Splunk search interface. The search bar contains the query `index = main sourcetype = vpn_logs`. The time range is set to `All time (real-time)`. The search results show 50 of 50 events matched. The results are displayed in a table with columns `Time` and `Event`. The events are clearly defined by the `CONNECT` and `DISCONNECT` actions, and the `sourcetype` is `vpn_logs`. A callout box labeled `Individual Events` points to the event entries.

Time	Event
12/1/25 3:51:55.000 AM	User: Emily Davis, Server: Server C, Action: DISCONNECT host = vpn_server : source = /opt/splunk/etc/apps/DataApp/bin/vpnlogs : sourcetype = vpn_logs
12/1/25 3:51:55.000 AM	User: Bob Johnson, Server: Server C, Action: CONNECT host = vpn_server : source = /opt/splunk/etc/apps/DataApp/bin/vpnlogs : sourcetype = vpn_logs
12/1/25 3:51:55.000 AM	User: Bob Johnson, Server: Server A, Action: CONNECT host = vpn_server : source = /opt/splunk/etc/apps/DataApp/bin/vpnlogs : sourcetype = vpn_logs
12/1/25 3:51:55.000 AM	User: Michael Brown, Server: Server C, Action: DISCONNECT host = vpn_server : source = /opt/splunk/etc/apps/DataApp/bin/vpnlogs : sourcetype = vpn_logs

Answer the questions below

Which configuration file is used to specify parsing rules?

props.conf

| What regex pattern is used in the case above to break the events?

(CONNECT|DISCONNECT)

Task 7 - Parsing Multi-Line Events

Understanding the Events: In this task, we will be utilizing the `authentication_logs` executable. Let's copy it over to our app's `/bin` directory and run it using the commands below.

- `cp /home/ubuntu/Downloads/scripts/authentication_logs /opt/splunk/etc/apps/DataApp/bin/`
- `/opt/splunk/etc/apps/DataApp/bin/authentication_logs`

This log clearly contains multiple lines and provides information about the authentication type, machine, timestamp, department, and other relevant details. Let's get started with ingestion and see how we can manage the events using Splunk's configuration files.

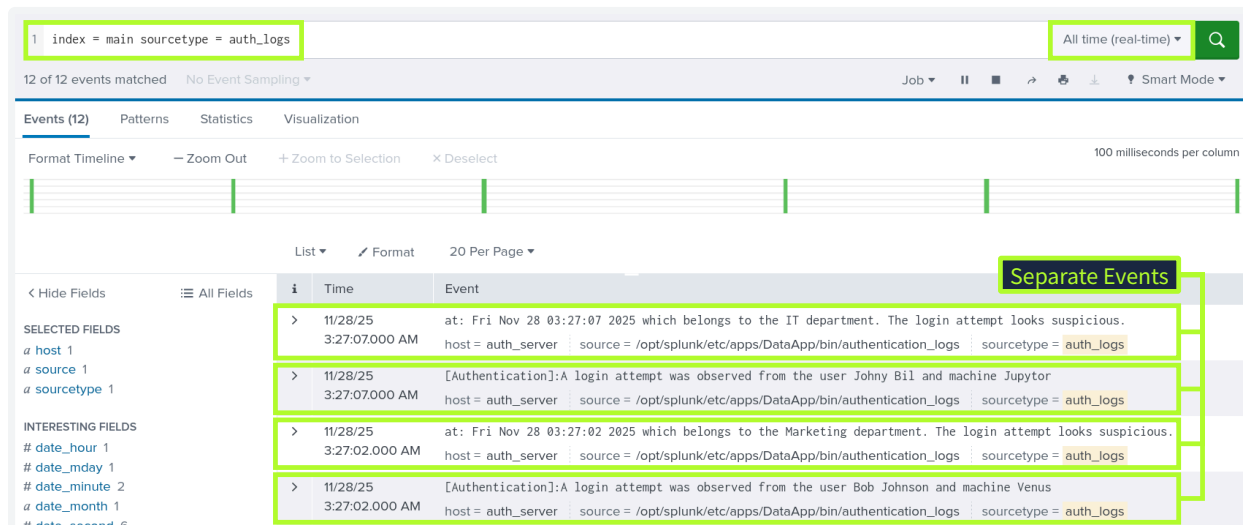
```
[Authentication]: A login attempt was observed from the user
Johnny Bil and machine Neptune
at: Fri Nov 28 09:06:53 2025 which belongs to the Development
department. The login attempt looks Normal.
```

Ingesting Authentication Logs: Just as we have done previously, we need to alter our `inputs.conf` file in the `/local` directory of our app for Splunk to know where the script is located. Add the entry below to specify the script location, interval, `index`, `sourcetype`, and `host`. Don't forget to restart Splunk after making the changes.

```
[script:///opt/splunk/etc/apps/DataApp/bin/authentication_logs]
index = main
sourcetype = auth_logs
host = auth_server
interval = 5
```

Checking Out the Logs: Back in the Splunk instance, head to your app and query `index = main sourcetype = auth_logs`. Ensure your time range is set to `All time`

(real time). The good news is, your logs are being ingested but Splunk is breaking each event into two separate events since it is unable to determine the boundaries. Let's fix this.



Fixing Event Boundaries: To address this issue, we will again be using the `props.conf` configuration file to alter our logs processing properties. We know from the previous screenshot that each log begins with the term `[Authentication]`, and we can use this to create a regex pattern and instruct Splunk how to split the events.

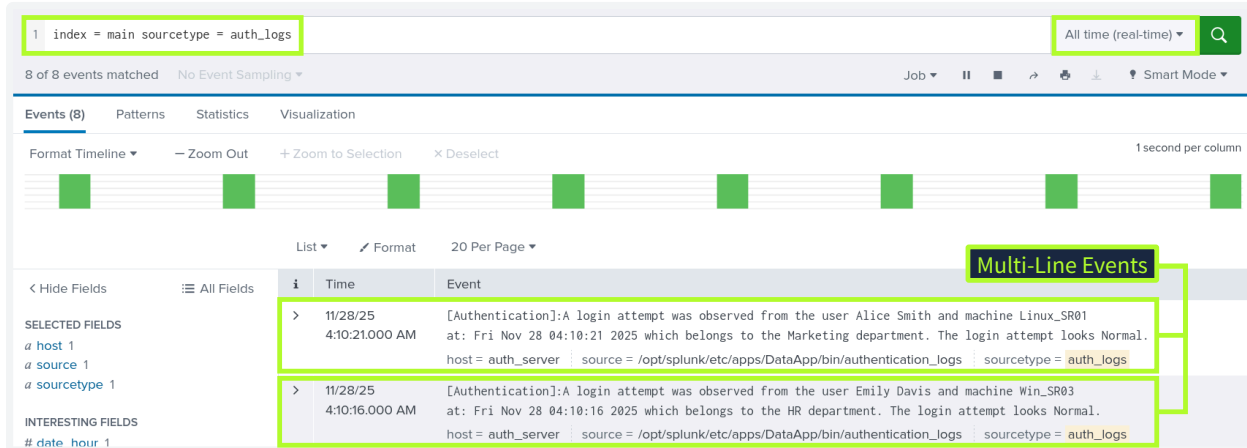
Let's go over the new stanza line by line.

- `[auth_logs]` Specifying the `sourcetype` we set in `inputs.conf`
- `SHOULD_LINEMERGE = true` Instructing Splunk that it should combine multiple lines into single events
- `BREAK_ONLY_BEFORE = \[Authentication\]` Break into a new event before the term `[Authentication]`

You can go ahead and add this stanza to your `props.conf` file in the `/local` directory, then restart Splunk.

```
[auth_logs]
SHOULD_LINEMERGE = true
BREAK_ONLY_BEFORE = \[Authentication\]
```

Verifying the Changes: Let's head back to the Splunk instance and check on the configuration changes we made. Again, use the query `index = main sourcetype = auth_logs` and the previously set time range.



Answer the questions below

Which configuration setting was used to ensure Splunk breaks the event boundary before the specified regex pattern?

BREAK_ONLY_BEFORE

Which regex pattern did you use in your props.conf file to specify where Splunk should break the events?

\[Authentication\]

Task 8 - Masking Sensitive Data

Masking sensitive fields, such as credit card numbers, is crucial for maintaining compliance with standards like the Payment Card Industry Data Security Standard (PCI DSS) and the Health Insurance Portability and Accountability Act (HIPAA). Splunk offers features such as field masking and anonymization to safeguard sensitive data.

Understanding the Events: Let's now use the final script in our `Downloads` folder. You can go ahead and copy it over now, then run it with the commands below.

- `cp /home/ubuntu/Downloads/scripts/purchase-details /opt/splunk/etc/apps/DataApp/bin/`

- `/opt/splunk/etc/apps/DataApp/bin/purchase-details`

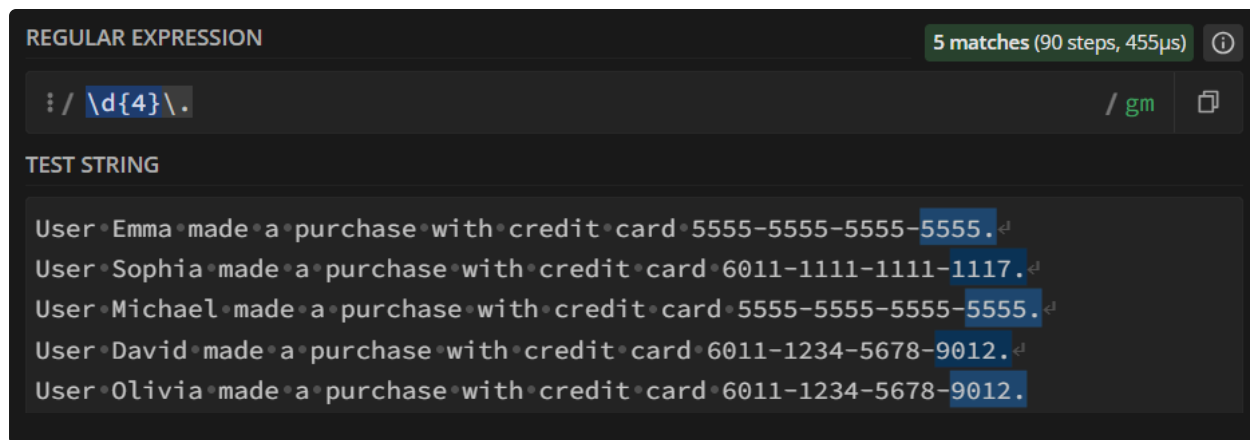
Based on the output, you can see that the logs track the user and credit card numbers used in purchases.

```
User Emma made a purchase with credit card 5555-5555-5555-5555.
User Sophia made a purchase with credit card 6011-1111-1111-1117.
User Michael made a purchase with credit card 5555-5555-5555-5555.
User David made a purchase with credit card 6011-1234-5678-9012.
User Olivia made a purchase with credit card 6011-1234-5678-9012.
```

Ingesting Purchase Logs: Let's get these logs into Splunk by altering our `inputs.conf` file. Add the stanza below.

```
[script:///opt/splunk/etc/apps/DataApp/bin/purchase-details]
index = main
sourcetype = purchase_logs
host = order_server
interval = 5
```

Setting Up Event Boundaries: Instead of jumping straight into Splunk, let's use what we learned previously about event boundaries and set up a regular expression to help Splunk identify the boundary of each event. Back at [regex101](#)(opens in new tab), insert the expression `\d{4}\.`, and the sample logs from above as the test string. This expression matches any digit from `0-9`, repeats the token four times, and matches the literal `.` character which is the end of each log line.



Altering Processing Properties: Now that we have a working regular expression, we must add a new stanza to `props.conf`.

- Add the following stanza to `/opt/splunk/etc/apps/DataApp/local/props.conf`
- Restart Splunk with `/opt/splunk/bin/splunk restart`

```
[purchase_logs]
SHOULD_LINEMERGE = true
MUST_BREAK_AFTER = \d{4}\.
```

Checking Out the Logs: Back in Splunk, run the search `index = main sourcetype = purchase_logs` and set your time range to `All time (real time)`. As shown below, and in your Splunk instance, the logs are indeed being ingested, and the event boundaries are correct. We have one issue, though: sensitive customer credit card information is unmasked for everyone to see. Let's fix it!

1 index = main sourcetype = purchase_logs

All time (real-time)

20 of 20 events matched No Event Sampling

Events (20) Patterns Statistics Visualization

Format Timeline Zoom Out Zoom to Selection Deselect 100 milliseconds per column

List Format 20 Per Page

	i	Time	Event
< Hide Fields			
SELECTION FIELDS			
a host 1			
a source 1			
a sourcetype 1			
INTERESTING FIELDS			
a index 1			
a punct 2			
a splunk_server 1			
a timestamp 1			

Masking Credit Card Information: To mask the sensitive data, we need to make some changes to our `props.conf` file. By utilizing the `sedcmd` setting in our previously made stanza, we can mask the last 12 digits of each credit card number that appear in the event data. This setting is based on the Unix command `sed` and allows us to use regular expression substitution to replace values with a masking pattern.

- Open up your `props.conf` file in `/local`
- Locate the previously made stanza for `purchase_logs`
- Add `SEDCMD-cc = s/-\d{4}-\d{4}-\d{4}/-XXXX-XXXX-XXXX/g` to your stanza
- Restart Splunk with `/opt/splunk/bin/splunk restart`

Your newly altered stanza should appear as follows.

```
[purchase_logs]
SHOULD_LINEMERGE = true
MUST_BREAK_AFTER = \d{4}\.
SEDCMD-cc = s/-\d{4}-\d{4}-\d{4}/-XXXX-XXXX-XXXX/g
```

Before examining the events in Splunk, let's briefly break down the above addition.

- `SEDCMD-cc` SED command setting with a name of our choosing. `cc` in this case
- `s/-\d{4}-\d{4}-\d{4}/-XXXX-XXXX-XXXX/g` The regex pattern we wish to match, along with the replacement values

Verifying the Changes: Back in your app, enter the query `index = main sourcetype = purchase_logs` and set your time range to `All time (real time)`. You have successfully used Splunk's configuration files to mask sensitive customer credit card information. Nicely done!

The screenshot shows the Splunk search results for the query `index = main sourcetype = purchase_logs`. The time range is set to `All time (real-time)`. The search results show 35 events. The first four events are displayed, each showing a purchase made by a user. The credit card numbers are masked with `XXXX-XXXX-XXXX-XXXX`. A yellow box highlights the masked credit card numbers, and a label `Masked Sensitive Data` points to them.

	Time	Event
>	12/1/25 4:19:56.000 AM	User Alex made a purchase with credit card 1234-XXXX-XXXX-XXXX. host = order_server source = /opt/splunk/etc/apps/DataApp/bin/purchase-details sourcetype = purchase_logs
>	12/1/25 4:19:56.000 AM	User John boy made a purchase with credit card 6011-XXXX-XXXX-XXXX. host = order_server source = /opt/splunk/etc/apps/DataApp/bin/purchase-details sourcetype = purchase_logs
>	12/1/25 4:19:56.000 AM	User Emma made a purchase with credit card 3056-XXXX-XXXX-XXXX. host = order_server source = /opt/splunk/etc/apps/DataApp/bin/purchase-details sourcetype = purchase_logs
>	12/1/25 4:19:56.000 AM	User Emma made a purchase with credit card 4012-XXXX-XXXX-XXXX. host = order_server source = /opt/splunk/etc/apps/DataApp/bin/purchase-details sourcetype = purchase_logs

Answer the questions below

Which payment security standard did you adhere to by masking sensitive customer credit card information?

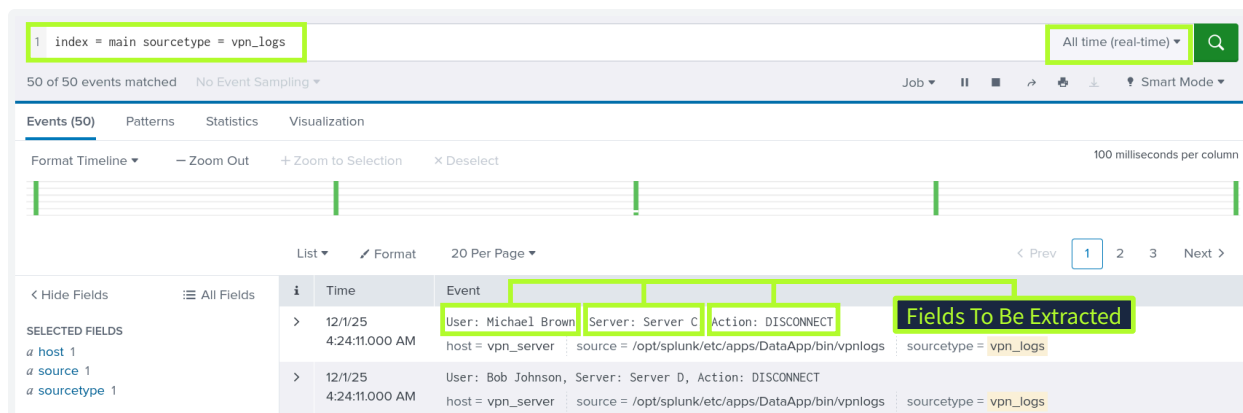
PCI DSS

What is the full value of the SEDCMD you used in props.conf?

`s/[-\d{4}-\d{4}-\d{4}]/-XXXX-XXXX-XXXX/g`

Task 9 - Extracting Custom Fields

Submit the query `index = main sourcetype = vpn_logs`, and let's take a look at the events. Each event has three distinct fields: `User`, `Server`, and `Action`. Let's dive into some of Splunk's other configuration files and begin our extraction.



Creating a Regex Pattern: Let's once again take a look at the sample event logs from the `vpnlogs` executable. We will use the sample logs to create a regex pattern that helps extract our three target fields. Back on [regex101](#)(opens in new tab), paste in the following regular expression, and the example logs below into the test string.

- `User:\s(.+?),\sServer:\s(.+?),\sAction:\s(\w+)`

```
User: John Doe, Server: Server B, Action: DISCONNECT
User: Bob Johnson, Server: Server A, Action: CONNECT
User: John Doe, Server: Server D, Action: CONNECT
User: Emily Davis, Server: Server C, Action: CONNECT
User: Bob Johnson, Server: Server B, Action: CONNECT
User: Alice Smith, Server: Server C, Action: DISCONNECT
User: John Doe, Server: Server D, Action: CONNECT
User: Alice Smith, Server: Server C, Action: CONNECT
User: Michael Brown, Server: Server B, Action: DISCONNECT
```

This pattern captures all three fields and organizes them into groups, as shown in the screenshot below.

REGULAR EXPRESSION 9 matches (616 steps, 395µs)

REGEX `/ User:\s(.+?),\sServer:\s(.+?),\sAction:\s(\w+)/g`

TEST STRING

```
User: John-Doe, Server: Server-B, Action: DISCONNECT
User: Bob-Johnson, Server: Server-A, Action: CONNECT
User: John-Doe, Server: Server-D, Action: CONNECT
User: Emily-Davis, Server: Server-C, Action: CONNECT
User: Bob-Johnson, Server: Server-B, Action: CONNECT
User: Alice-Smith, Server: Server-C, Action: DISCONNECT
User: John-Doe, Server: Server-D, Action: CONNECT
User: Alice-Smith, Server: Server-C, Action: CONNECT
User: Michael-Brown, Server: Server-B, Action: DISCONNECT
```

EXPLANATION

- `/` matches the characters `/`: literally (case sensitive)
- matches any whitespace character (equivalent to `[\r\n\t\f\v ]`)
- `1st Capturing Group (.+?)`
 - `.` matches any character (except for line terminators)
 - `+` matches the previous token between **one** and **unlimited** times, as few times as possible, expanding as needed (lazy)
 - `?` matches the character `.` with index **44**... (**?** **C**... or **S**...) literally (case sensitive)

MATCH INFORMATION

Match	Index	Value
Match 1	0-52	User: John-Doe, Server: Server-B, Action: DISCONNECT
Group 1	6-14	John-Doe
Group 2	24-32	Server-B
Group 3	42-52	DISCONNECT

SUBSTITUTION success (65µs)

insert your replacement value here

Configuring Transformations: As discussed in Task 4, `transforms.conf` allows you to define data transformations and enrichments on indexed events. Let's create this new file in the `/local` directory of our app and paste in the stanza below.

```
[vpn_custom_fields]
REGEX = User:\s(.+?),\sServer:\s(.+?),\sAction:\s(\w+)
FORMAT = User::$1 Server::$2 Action::$3
WRITE_META = true
```

With this entry, we assign a name to our field extraction, `vpn_custom_fields`, specify the regex pattern, and provide the format so that Splunk knows how to name the extracted fields.

Updating the Processing Properties: The next step is to update your `props.conf` file. This is as simple as adding a line to our `vpn_logs` stanza to specify the name of our field extraction. Your updated `props.conf` stanza should appear as follows. Note how we are referencing the `vpn_custom_fields` naming convention from `transforms.conf`.

```
[vpn_logs]
SHOULD_LINEMERGE = false
MUST_BREAK_AFTER = (CONNECT|DISCONNECT)
TRANSFORM-vpn = vpn_custom_fields
```

Configuring Fields: The last step is to create a `fields.conf` file in your `/local` directory. We previously discussed that this configuration file defines

how custom or extracted fields behave in Splunk. With this entry, we instruct Splunk to treat our extracted values as indexed fields, ensuring they are recognized and searchable. Go ahead and create `fields.conf`, add the entries below, and restart Splunk with `/opt/splunk/bin/splunk restart` and jump back into your DataApp!

```
[User]
INDEXED = true
```

```
[Server]
INDEXED = true
```

```
[Action]
INDEXED = true
```

Verifying Field Extraction: Let's run the same query as before and see how our event data is looking.

- `index = main sourcetype = vpn_logs`

Awesome! You have successfully performed custom field extraction using a range of Splunk configuration files. Now your events are searchable by these fields.

The screenshot shows the Splunk search interface. The search bar at the top contains the query `index = main sourcetype = vpn_logs`. Below the search bar, it indicates "90 of 90 events matched". The results are displayed in a table with columns for Time and Event. The first event shows "User: Bob Johnson, Server: Server E, Action: CONNECT". The second event shows "Action = CONNECT | Server = Server E | User = Bob Johnson". A box labeled "Extracted Fields" points to the fields in the results table.

Purchase Log Field Extraction: In the previous task, we ingested purchase logs and masked the sensitive customer credit card information. Your next task is to configure Splunk to extract the `User` and `CC_Number` fields from those events.

- Start with `transforms.conf` and add an entry for your field extraction using the regular expression below `^User\s+([A-Za-z]+(?:\s+[A-Za-z]+)*)\s+made a purchase with credit card\s+([0-9]{4}(?:-XXXX){3}|(?:\d{4}[\s-]){3}\d{4})\.`
- Next, update your `props.conf` configuration file
- Alter `fields.conf` to introduce your new fields
- Restart Splunk

Answer the questions below

After performing field extraction on the purchase logs, how many User field values exist within the event data?

14

How many unique masked CC_Number field values were extracted from the events?

10

Task 10 - Conclusion

Splunk data processing, covering:

- **Core Skills:** Parsing raw data, defining event boundaries, handling multi-line logs, masking sensitive information, and extracting custom fields.
- **Configuration:** Working with configuration files and creating basic Splunk apps.
- **SOC Relevance:** These abilities are essential for **SOC analysts** to build cleaner indexes and improve environmental visibility.